

Python OOP + Django REST Framework Serializer

Learning Cheat Sheet — based on your questions and examples

1. The Big Picture

A DRF serializer is a Python **class**. Your serializer inherits functionality from DRF's `ModelSerializer`.

```
class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = "__all__"
```

Mental model: Python OOP → Class → Inheritance → `ModelSerializer` → Your Serializer → DRF automatically uses certain methods.

2. Class, Object, and Method

Class: a blueprint. **Object/instance:** a concrete object created from the class. **Method:** a function defined inside a class.

```
class StudentSerializer(serializers.ModelSerializer):
    def hello(self):
        return "Hello"

serializer = StudentSerializer(student)
serializer.hello()
```

3. What is self?

self refers to the current object/instance. So `self.hello()` means: call the `hello()` method belonging to this serializer object.

```
class StudentSerializer(serializers.ModelSerializer):
    def hello(self):
        return "Hello"

    def test(self):
        return self.hello()
```

4. Inheritance ■

```
class StudentSerializer(serializers.ModelSerializer):
    ...
```

`StudentSerializer` inherits from `ModelSerializer`. Therefore it gets behavior supplied by the parent class and can customize selected behavior.

Conceptually: `StudentSerializer` → `ModelSerializer` → `Serializer`.

5. Method Overriding ■■

If the parent class already provides a method, your subclass can define a method with the same name and replace/customize that behavior for the subclass.

```
class StudentSerializer(serializers.ModelSerializer):
    def create(self, validated_data):
        # your custom creation logic
    ...
```

This is called **method overriding**. The same idea applies to `create()`, `update()`, `validate()`, and `to_representation()`.

6. DRF Hooks ■■■■

Some method names have special meaning because DRF knows those names and calls them at particular stages.

Method	Purpose
<code>create()</code>	Create a model instance when saving new data.
<code>update()</code>	Update an existing model instance.

validate()	Object-level / multi-field validation.
to_representation()	Customize object → serialized output.
validate_()	Validate one particular serializer field.

7. Can You Create Your Own Methods?

Yes. You can create any method you want. DRF will not automatically call it unless you explicitly call it yourself.

```
class StudentSerializer(serializers.ModelSerializer):
    def calculate_status(self, age):
        if age >= 18:
            return "Adult"
        return "Minor"

# You call it yourself:
serializer.calculate_status(20)
```

Remember: create(), update(), validate(), and to_representation() are recognized DRF methods. calculate_status(), hello(), etc. are your own methods.

8. validate_()

DRF uses the naming pattern **validate_ + field name** for field-level validation.

```
def validate_age(self, value):
    if value < 18:
        raise serializers.ValidationError(
            "Student age must be 18 or above."
        )
    return value

def validate_name(self, value):
    ...
```

For a field named age → validate_age(). For name → validate_name(). For email → validate_email(). The naming pattern matters.

9. Field Validation vs Object Validation

```
def validate_age(self, value):
    # ONE field
    ...

def validate(self, attrs):
    # MULTIPLE fields together
    if attrs["course"] == "BCA" and attrs["age"] < 18:
        raise serializers.ValidationError(
            "BCA student must be 18 or above."
        )
    return attrs
```

validate_age() → one field. **validate()** → multiple fields / relationships between fields.

10. create()

When valid serializer data is saved for a new object, DRF can use create(). You can override it to customize how the model is created.

```
def create(self, validated_data):
    student = Student.objects.create(**validated_data)
    return student
```

Typical flow: POST → validation → serializer.save() → create() → database.

11. update()

When an existing object is updated, DRF can use update().

```
def update(self, instance, validated_data):
    instance.name = validated_data.get("name", instance.name)
```

```
instance.save()
return instance
```

Easy memory rule: **CREATE** → **create()**; **UPDATE** → **update()**.

12. to_representation()

This method controls/customizes how a model/object becomes serialized output.

```
def to_representation(self, instance):
    data = super().to_representation(instance)
    data["message"] = "Hello"
    return data
```

Mental model: database object → to_representation() → Python dictionary → API response.

13. super() ■

super() lets you access behavior from the parent class. Calling super().to_representation(instance) runs the parent implementation first, then you can modify its result.

```
data = super().to_representation(instance)
data["extra"] = "something"
return data
```

Mental model: Parent implementation → super() → your customization.

14. Serializer context

DRF can provide extra information to a serializer through **self.context**. A request is commonly available when the serializer is created by a DRF view.

```
req = self.context.get("request")

print(req.user)
print(req.method)
```

Conceptually: Serializer → self.context → request → user / method.

15. .get() vs [] — Normal Python Dictionary Behavior

```
request = self.context.get("request")
# Missing key → None

request = self.context["request"]
# Missing key → KeyError
```

This difference is not specific to DRF; it is normal Python dictionary behavior.

16. Writable Nested Serializer

```
class EmployeeWritableNestedSerializer(serializers.ModelSerializer):
    department = DepartmentSerializer()

    class Meta:
        model = Employee
        fields = "__all__"
```

A nested serializer represents related/nested data. For writable nested data, custom create() / update() logic may be needed.

17. Nested create() — Your Example

```
def create(self, validated_data):
    departmentdata = validated_data.pop("department")

    department = Department.objects.create(
        **departmentdata
    )

    employee = Employee.objects.create(
        department=department,
        **validated_data
```

```
)

return employee
```

Mental model: validated_data → separate department data → create Department → create Employee → connect Employee to Department.

18. Nested update() — Your Example

```
def update(self, instance, validated_data):
    departmentdata = validated_data.pop("department", None)

    instance.name = validated_data.get("name", instance.name)
    instance.salary = validated_data.get("salary", instance.salary)

    if departmentdata:
        instance.department.name = departmentdata.get(
            "name", instance.department.name
        )
        instance.department.save()

    instance.save()
    return instance
```

This custom update lets you update both the Employee and its nested Department.

19. OOP Concepts in Your DRF Code

Your code	OOP concept
class StudentSerializer	Class
StudentSerializer(...)	Object / instance
self	Current object
ModelSerializer	Parent/base class
class X(ModelSerializer)	Inheritance
Custom create()	Method overriding
Custom update()	Method overriding
super()	Access parent implementation
calculate_status()	Your own method
Same method name, different implementation	Polymorphism

20. Polymorphism

```
class StudentSerializer(serializers.ModelSerializer):
    def create(self, validated_data):
        # Student creation logic
        ...

class EmployeeSerializer(serializers.ModelSerializer):
    def create(self, validated_data):
        # Employee creation logic
        ...
```

Both classes expose create(), but each can behave differently. Basic idea: **same interface/name** → **different behavior**.

21. The Most Important Mental Model ■■■

Don't just memorize that create() is for POST. Understand the framework mechanism:

```
DRF has a serializer system
↓
```


Class + Inheritance + Methods + Overriding + super() + Framework Hooks = a large part of what you are seeing in DRF serializers.

If you understand these OOP ideas, DRF serializer methods become much easier to understand instead of being a list of names to memorize.